

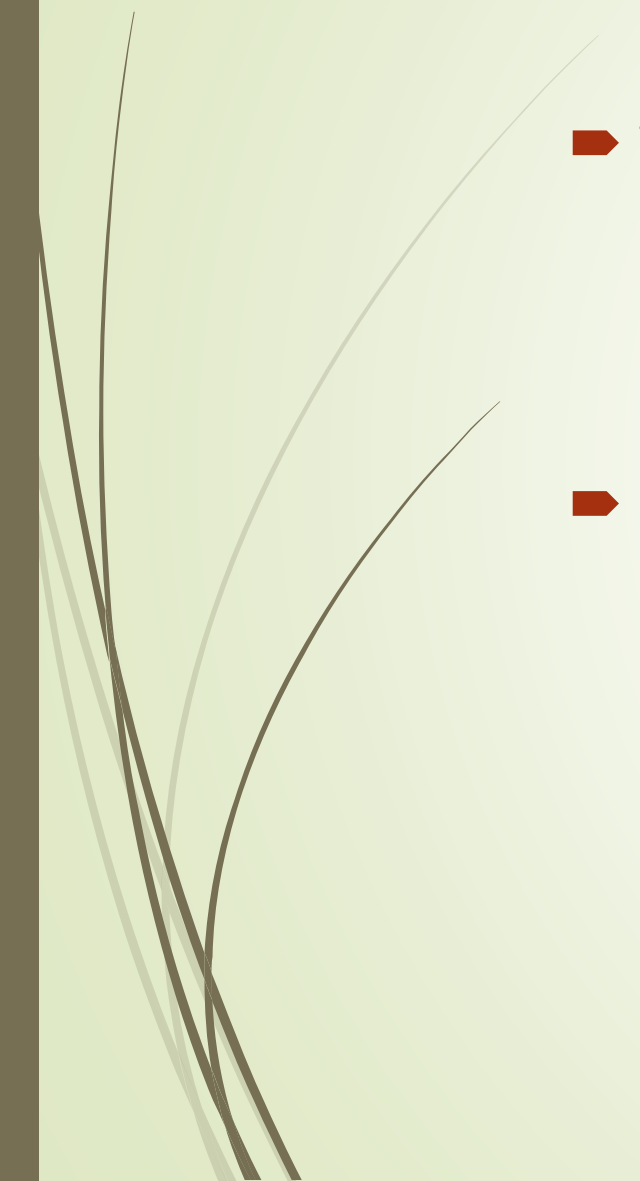
DATA STRUCTURES

Complexity Theory-II





Constant Factors

- ▶ The growth rate is not affected by
 - ▶ Constant factors or
 - ▶ Lower-order terms
 - ▶ Examples
 - ▶ $10^2n + 10^5$ is a linear function
 - ▶ $10^5n^2 + 10^8n$ is a quadratic function
- 

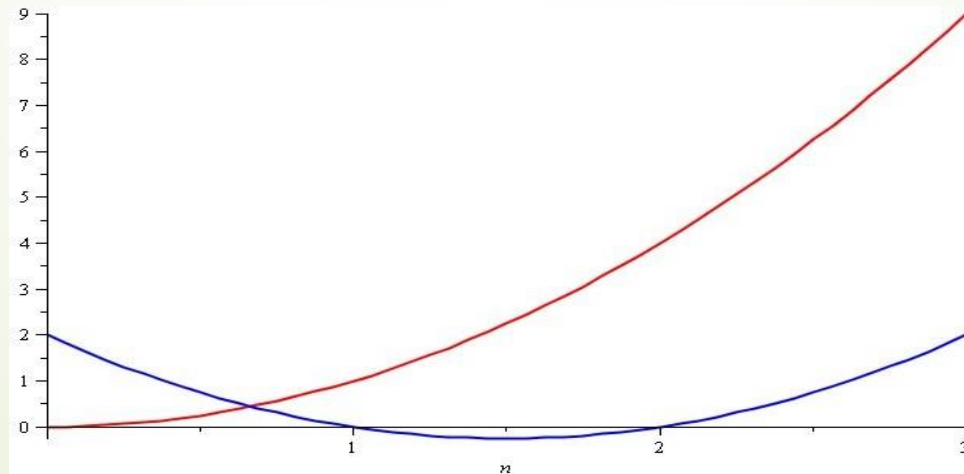
Growth Rate – Example

- Consider the two functions

$$f(n) = n^2$$

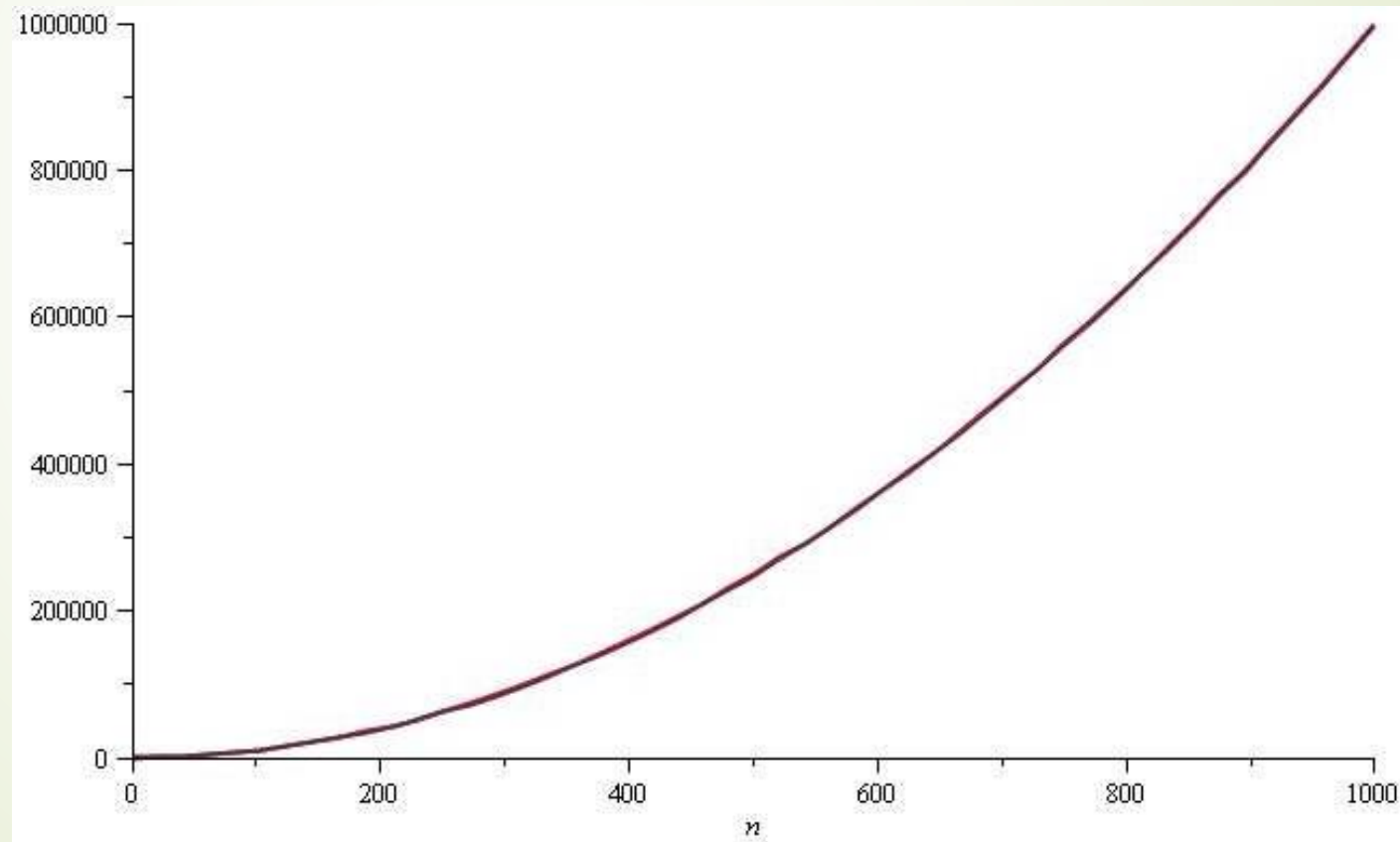
$$g(n) = n^2 - 3n + 2$$

- Around $n = 0$, they look very different



Growth Rate – Example

- Yet on the range $n = [0, 1000]$, $f(n)$ and $g(n)$ are (relatively) indistinguishable



Growth Rate – Example

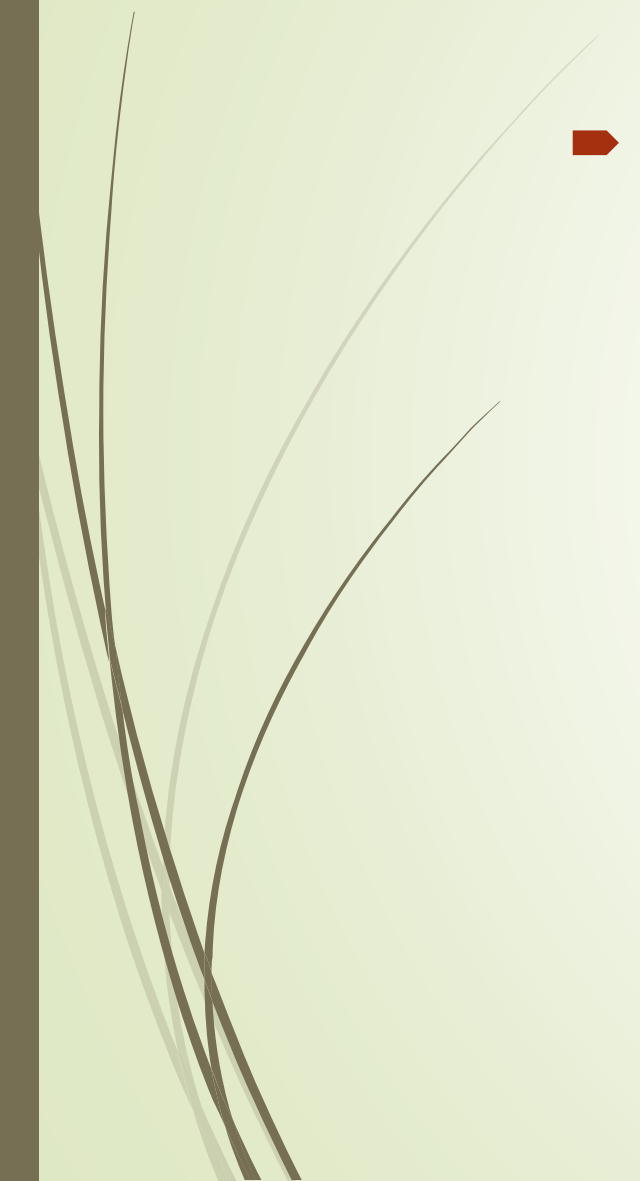
- The absolute difference is large, for example,
 - $f(1000)=1000000$
 - $g(1000)=997002$
- But the relative difference is very small

$$\left| \frac{f(1000) - g(1000)}{f(1000)} \right| = 0.002998 < 0.3\%$$

- The difference goes to zero as $n \rightarrow \infty$

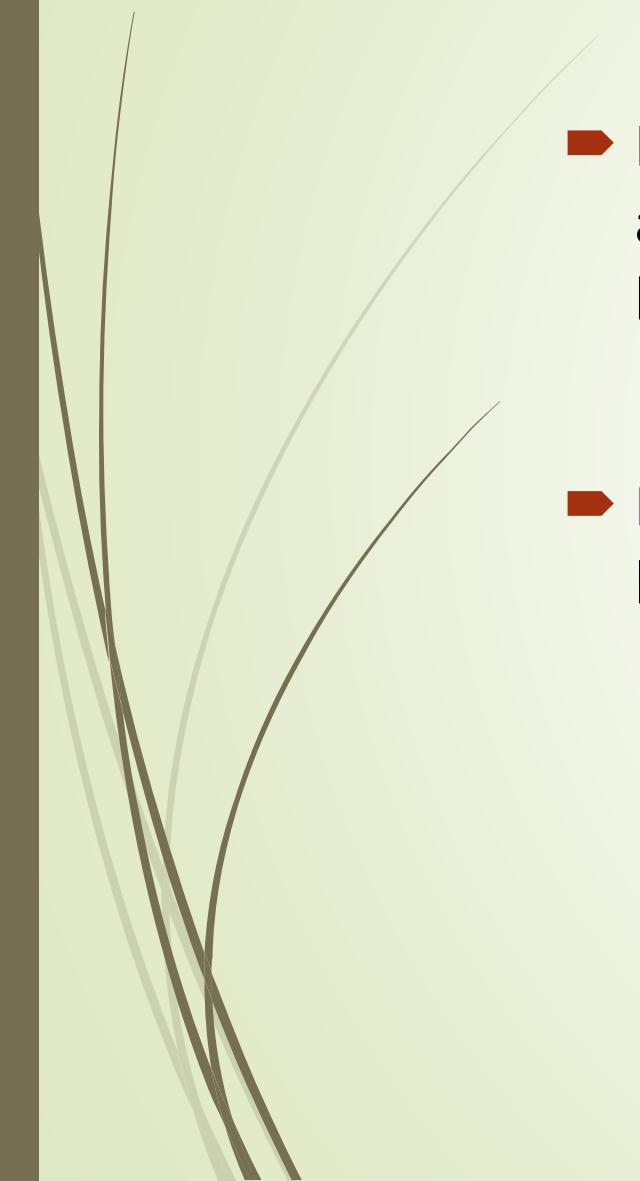


Constant Factors

- How do we get rid of the constant factors to focus on the essential part of the running time?
 - Asymptotic Analysis
- 

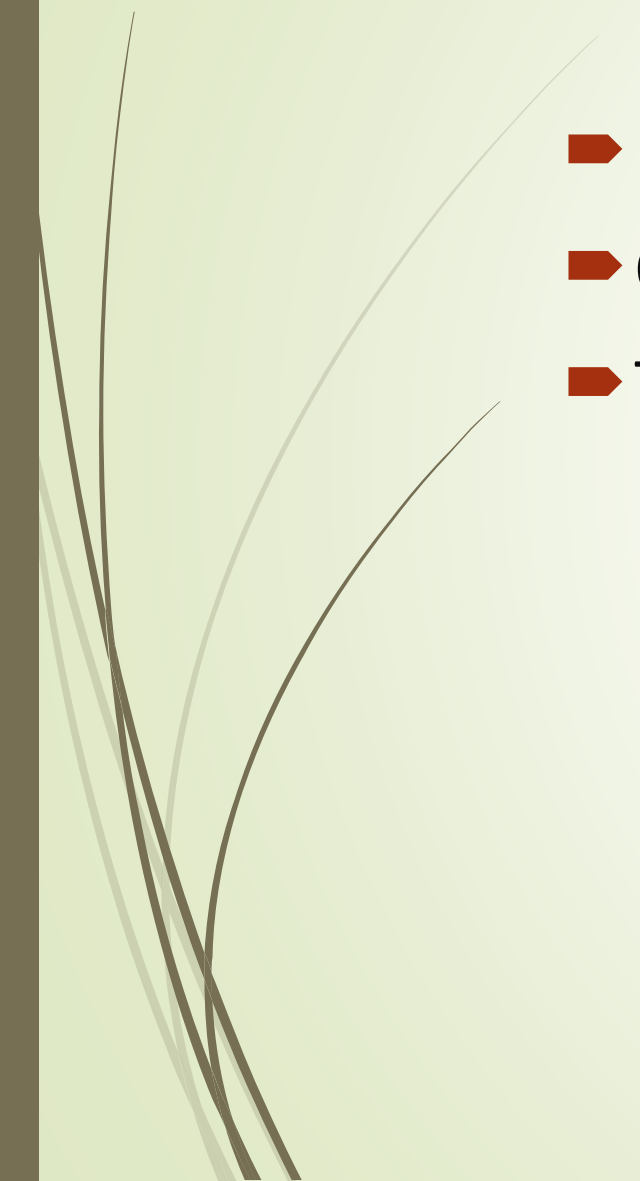


Asymptotic Analysis

- In computer science and applied mathematics, particularly the analysis of algorithms, asymptotic analysis is a method of describing limiting behavior.
 - Examples The performance of algorithms when applied to very large input data
- 



Comparing Functions: Asymptotic Notation

- Big Oh Notation: Upper bound (worst case)
 - Omega Notation: Lower bound (best case)
 - Theta Notation: Tighter bound (average case)
- 



Upper Bound – Big-Oh Notation

- Indicates the upper or highest growth rate that the algorithm can have
 - Ignore constant factors and lower order terms
 - Focus on main components of a function which affect its growth

Upper Bound – Big-Oh Notation

- If $f(N)$ and $g(N)$ are two complexity functions, we say

$$f(n) = O(g(n))$$

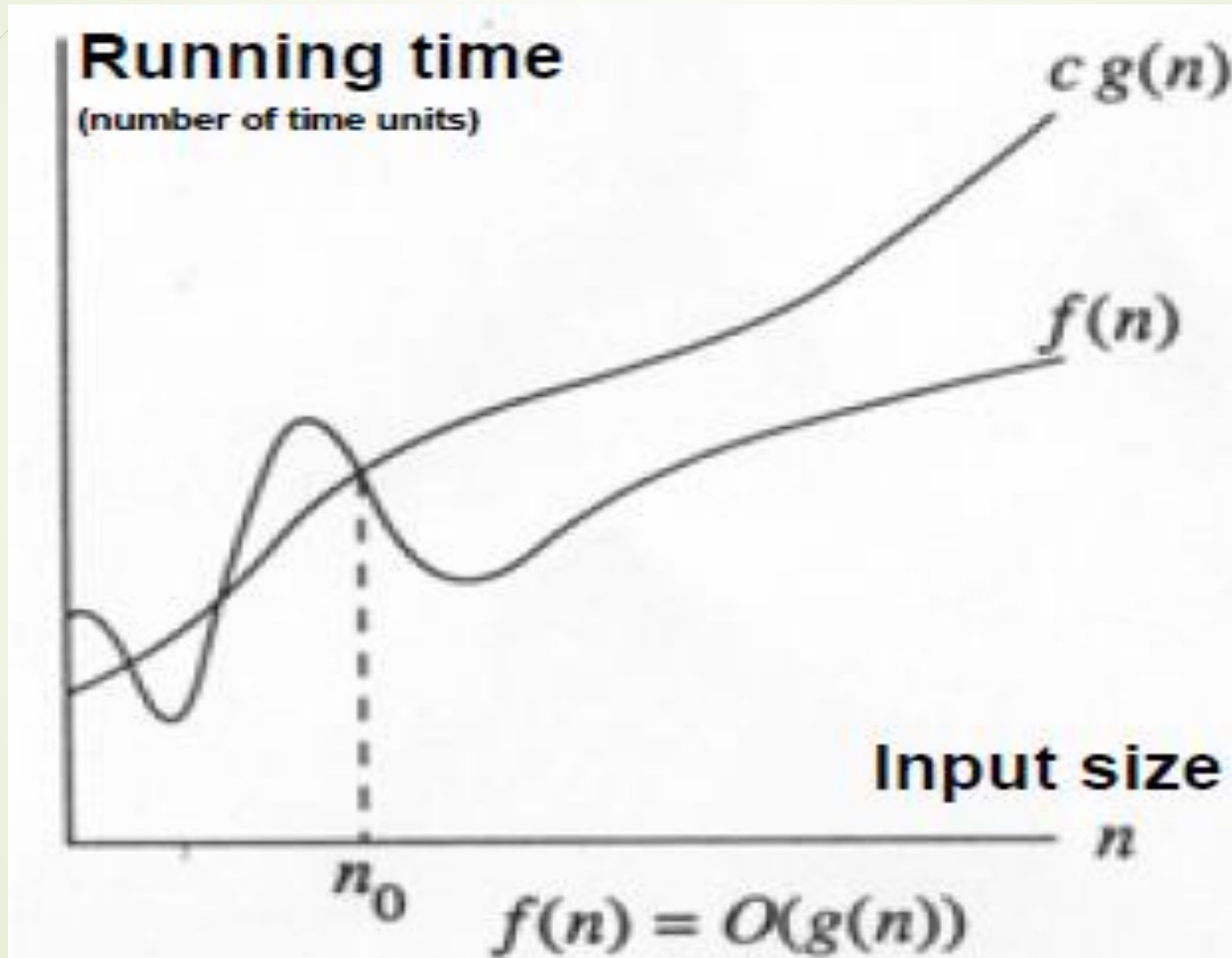
- (read "f(n) is big-O of g(n)")

- There are positive constants c and n_0 such that

$$f(n) \leq c g(n) \text{ when } n \geq n_0$$

- The growth rate of $f(n)$ is less than or equal to the growth rate of $g(n)$. In other words, $f(n)$ grows no faster than $g(n)$ for large N .
- $g(n)$ is an upper bound on $f(n)$.

Growth Rate of the Algorithm Running Time





Big-Oh Notation Rules

- ▶ When considering the growth rate of a function using Big-Oh notation,
 - ▶ we ignore the lower order terms
 - ▶ We ignore the coefficients of the highest-order term; and
 - ▶ we don't need to specify the base of logarithm
 - ▶ Note that changing the base from one constant to another changes the value of the logarithm by only a constant factor

Example

- $N^2 / 2 - 3N = O(N^2)$
- (We throw away leading constants and low-order terms.)
- $1 + 4N = O(N)$
- $7N^2 + 10N + 3 = O(N^2)$
- $\log N + N = O(N)$

$$\log_{10} N = \log_2 N / \log_2 10 = O(\log_2 N) = O(\log N)$$

Big-Oh Notation

- Example: $2n + 10$ is $O(n)$
 - $2n + 10 \leq cn$
 - $10 \leq cn - 2n$
 - $(c - 2)n \geq 10$
 - $n \geq 10/(c - 2)$
 - Pick $c = 3$ and $n_0 = 10$

Big-Oh Example

- Example: the function n^2 is not $O(n)$

$$n^2 \leq cn$$

$$n \leq c$$

- The above inequality cannot be satisfied since c must be a constant and n can take any non-negative integral value

Big-Oh Examples

➤ **$7n-2$ is $O(n)$**

need to find two positive constants: c , n_0 , such that for all $n \geq n_0$

$$7n-2 \leq c \cdot n$$

$$c = 7 \text{ and } n_0 = 1$$

➤ **$3n^3 + 20n^2 + 5$ is $O(n^3)$**

need to find two positive constants: c , n_0 , such that for all $n \geq n_0$

$$3n^3 + 20n^2 + 5 \leq c \cdot n^3$$

$$c = 4 \text{ and } n_0 = 21$$

➤ **$3 \log n + \log \log n$ is $O(\log n)$**

need to find two positive constants: c , n_0 , such that

$$3 \log n + \log \log n \leq c \cdot \log n \text{ for all } n \geq n_0$$

this is true for $c = 4$ and $n_0 = 2$

Analyzing an Algorithm

- Simple Assignment

- $a = b$
- $O(1)$

- Simple loops

- `for(i=0; i<n; i++) { s; }`
- $O(n)$

- Nested loops

- `for(i=0; i<n; i++)`
 - `for(j=0; j<n; j++) { s; }`
- $O(n^2)$

Analyzing an Algorithm

- Loop index doesn't vary linearly

```
h=1;  
while(h<=n){  
    s;  
    h=2*h;  
}
```

h takes values 1, 2, 4, ... until it exceeds n

There are $1 + \log_2 n$ iterations

$O(\log_2 n)$



Standard Analysis Techniques

- Constant time statements
 - Analyzing Loops
 - Analyzing Nested Loops
 - Analyzing Sequence of Statements
 - Analyzing Conditional Statements
- 

Constant time statements

- Simplest case: $O(1)$ time statements
- Assignment statements of simple data types
`int x = y;`
- Arithmetic operations:
`x = 5 * y + 4 - z;`
- Array assignment:
`A[j] = 5;`
- Most conditional tests:
`if (x < 12) ...`

Analyzing Loops

- ▶ Any loop has two parts:
 - ▶ How many iterations are performed?
 - ▶ How many steps per iteration?
- ```
int sum = 0,j;
for (j=0; j < N; j++)
 sum = sum +j;
```
- ▶ Loop executes N times (0..N-1)
  - ▶ 4 = O(1) steps per iteration
- ▶ Total time is  $N * O(1) = O(N*1) = O(N)$

# Analyzing Nested Loops

- Treat just like a single loop and evaluate each level of nesting as needed:

```
int j,k;
for (j=0; j<N; j++)
 for (k=N; k>0; k--)
 sum += k+j;
```

- Start with outer loop:
  - How many iterations?  $N$
  - How much time per iteration? Need to evaluate inner loop
- Inner loop uses  $O(N)$  time
- Total time is  $N * O(N) = O(N*N) = O(N^2)$

# Analyzing Sequence of Statements

- For a sequence of statements, compute their complexity functions individually and add them up

```
for (j=0; j < N; j++)
 for (k =0; k < j; k++)
 sum = sum + j*k;
for (l=0; l < N; l++)
 sum = sum -l;
cout<<"Sum="<<sum;
```

- Total cost is  $O(N^2) + O(N) + O(1) = O(N^2)$

# Analyzing Conditional Statements

- What about conditional statements such as

```
if (condition)
 statement1;
else
 statement2;
```

- Running time = never more than the running time of the test Condition plus the larger of the running times of Statement1 and Statement2.



# Relatives of Big-Oh

## ➤ Big-Omega

- $f(n)$  is  $\Omega(g(n))$  is asymptotically greater than or equal to  $g(n)$ 
  - If there is a constant  $c > 0$  and an integer constant  $n_0 \geq 1$
  - Such that  $f(n) \geq c \cdot g(n)$  for  $n \geq n_0$

## ➤ Big-Theta

- $f(n)$  is  $\Theta(g(n))$  is asymptotically equal to  $g(n)$ 
  - if there are constants  $c_1 > 0$  and  $c_2 > 0$  and an integer constant  $n_0 \geq 1$
  - such that  $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$  for  $n \geq n_0$

# Final Notes

- Even though in this course we focus on the asymptotic growth using big-Oh notation, practitioners do care about constant factors occasionally
- Suppose we have 2 algorithms
  - Algorithm A has running time  $30000n$
  - Algorithm B has running time  $3n^2$
- Asymptotically, algorithm A is better than algorithm B
- However, if the problem size you deal with is always less than 10000, then the quadratic one is faster

